

Шифры перестановки

Хамза хуссен

16 марта 2026, Москва, Россия

Российский Университет Дружбы Народов

Цель работы

Изучение и практическое применение методов программной реализации шифров перестановки

Задание

1. Реализовать Маршрутное шифрование
2. Реализовать Таблицу Виженера.

Теоретическое введение

Маршрутное шифрование

1. Например, для шифрования текста нельзя недооценивать противника, разобьем его на блоки длины $n = 6$. Блоков получится $t = 5$. К последнему блоку припишем букву а. В качестве пароля выберем слово пароль. Теперь будем выписывать буквы по столбцам в соответствии с алфавитным порядком букв пароля и получим следующую криптограмму:
ЕНПНЗОАТАБОВОКННЕЬВЯЦТИА.

Н	Е	Л	Ь	З	Я
Н	Е	Д	О	О	Ц
Е	Н	И	В	А	Т
Ь	П	Р	О	Т	И
В	Н	И	К	А	А
П	А	Р	О	Л	Ь

Таблица Виженера

2. Открытый текст разбивается на блоки длины p . Ключ представляет собой последовательность из p натуральных чисел: $a_1, a_2, \dots, a_p$. Далее в каждом блоке первая буква циклически сдвигается вправо по алфавиту на a_1 позиций, вторая буква - на a_2 позиций, последняя - на a_p позиций. Для лучшего запоминания в качестве ключа можно взять осмысленное слово, а алфавитные номера входящих в него букв использовать для осуществления сдвигов. Например открытый текст CRYPTOGRAPHY и ключ MATH, получим : ORRWFOZYMPAF

C	R	Y	P	T	O	G	R	A	P	H	Y
M	A	T	H	M	A	T	H	M	A	T	H

Таблица Виженера

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z																										
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M

Выполнение лабораторной работы

Маршрутное шифрование

```
import math

ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

def normalize(text: str) -> str:
    text = text.upper()
    return "".join(ch for ch in text if ch in ALPHABET)

def column_order(key: str):
    """
    Returns column indices in the order they should be read,
    based on alphabetical sorting of key letters.
    """
    key = key.upper()
    indexed = list(enumerate(key)) # [(0, 'Z'), (1, 'E'), ...]
    indexed_sorted = sorted(indexed, key=lambda x: (x[1], x[0]))
    return [idx for idx, _ in indexed_sorted]
```

Маршрутное шифрование

```
def columnar_encrypt(plaintext: str, key: str, pad: str = "X") -> str:
    pt = normalize(plaintext)
    key = normalize(key)

    cols = len(key)
    rows = math.ceil(len(pt) / cols)
    pt = pt.ljust(rows * cols, pad)

    # Fill row by row
    grid = [list(pt[r*cols:(r+1)*cols]) for r in range(rows)]
    print("grid: ", grid)

    # Read columns in key-sorted order
    order = column_order(key)
    print("order: ", order)
    out = []
    for c in order:
        for r in range(rows):
            out.append(grid[r][c])
    return "".join(out)
```

Рис. 1: шифрование

Cipher: SUEAEYWTNSTEENDTENMOEITEUTRMHM

Маршрутное шифрование

```
def columnar_decrypt(ciphertext: str, key: str) -> str:
    ct = normalize(ciphertext)
    key = normalize(key)

    cols = len(key)
    rows = len(ct) // cols
    order = column_order(key)

    # Create empty grid
    grid = ["" for _ in range(cols)] for _ in range(rows)]

    # Fill columns in the same order they were read
    pos = 0
    for c in order:
        for r in range(rows):
            grid[r][c] = ct[pos]
            pos += 1

    # Read row by row
    return "".join("".join(row) for row in grid)
```

Рис. 2: дешифрование

Back : WEMUSTNOTUNDERESTIMATETHEENEMY

шифрование Виженера

```
def vigenere_encrypt(plaintext: str, key: str) -> str:
    key = normalize(key)
    out = []
    j = 0 # index in key (only advances when we encrypt a letter)

    for ch in plaintext:
        up = ch.upper()
        if up in ALPHABET:
            p = ALPHABET.index(up)
            k = ALPHABET.index(key[j % len(key)])
            c = ALPHABET[(p + k) % M]
            out.append(c)
            j += 1

    return "".join(out)
```

```
text = "CRYPTOGRAPHY"
key = "MATH"
enc = vigenere_encrypt(text, key)
```

```
print("Plain :", text)
print("Key   :", key)
print("Enc   :", enc)
```

```
Plain : CRYPTOGRAPHY
Key   : MATH
Enc   : ORRWFOZYMPAF
```

Рис. 3: шифрование

шифрование Виженера

```
def vigenere_decrypt(ciphertext: str, key: str) -> str:
    key = normalize(key)

    out = []
    j = 0

    for ch in ciphertext:
        up = ch.upper()
        if up in ALPHABET:
            c = ALPHABET.index(up)
            k = ALPHABET.index(key[j % len(key)])
            p = ALPHABET[(c - k) % M]
            out.append(p if ch.isupper() else p.lower())
            j += 1

    return "".join(out)

dec = vigenere_decrypt(enc, key)

print("Dec :", dec)

Dec : CRYPTOGRAPHY
```

Рис. 4: дешифрование

Выводы

Изученил и разработал методы программной реализации шифров перестановки.

Список литературы

1] https://en.wikipedia.org/wiki/Transposition_cipher

[2] https://en.wikipedia.org/wiki/Vigen%C3%A8re_cipher